



YamuLK

A Smart Travel Planning Web Application
for Sri Lankan Domestic Travellers

SYSTEM DESIGN REPORT

Menus • GUI Elements • Navigation • Back-End Plan • System Model

Platform	Responsive browser-based web application (desktop-first)
Technology Stack	MERN — MongoDB, Express.js, React.js, Node.js
Document	Lab 03 deliverable, continuing the Lab 02 task analysis

1. Introduction and Scope

YamuLK is a web application — it runs entirely in a desktop or laptop browser and is delivered as a single-page application built with React.js. It is not a mobile app: there is no phone frame, no bottom tab bar and no native device packaging. All interaction is designed for a wide viewport with a persistent top navigation bar, multi-column grids, sidebars, modals and hover states, and the layout degrades gracefully to tablet width using CSS media queries.

The system is implemented on the MERN stack: MongoDB for document storage, Express.js for the REST API, React.js for the browser client and Node.js as the server runtime. This report defines the menu structure, the GUI element inventory, the navigation model, the back-end plan and the overall system model that connect the ten interfaces designed earlier.

1.1 Technology Stack Summary

Layer	Technology	Responsibility in YamuLK
Front-End	React.js 18, React Router, Axios, Context API / Redux Toolkit, CSS Modules	Renders all ten screens as routed components, holds session state, calls the REST API and updates the view without page reloads.
Back-End	Node.js 20, Express.js 4, JWT, bcrypt, express-validator	Exposes REST endpoints, authenticates users, validates input, runs itinerary and budget logic, and proxies external APIs.
Database	MongoDB Atlas with Mongoose ODM	Stores users, destinations, trips, budgets, saved items and reviews as collections of JSON-like documents with schema validation.
External APIs	Google Maps Directions and Places, OpenWeather, Cloudinary	Supply routing and distance data, weather forecasts and hosted destination imagery.
Deployment	Vercel or Netlify (client), Render or Railway (server), MongoDB Atlas (database)	Client build served from a CDN; API served over HTTPS; database hosted as a managed cluster.

2. Menus and Menu Structure

Menus in a web application are persistent rather than modal. YamuLK uses a fixed top navigation bar that is visible on every authenticated page, a right-aligned account dropdown, an in-page category filter menu on the dashboard, and contextual action menus inside individual pages. The diagram shows the menu hierarchy; the table lists every menu, its items, who can see it and the action it triggers.

2.1 Menu Hierarchy Diagram

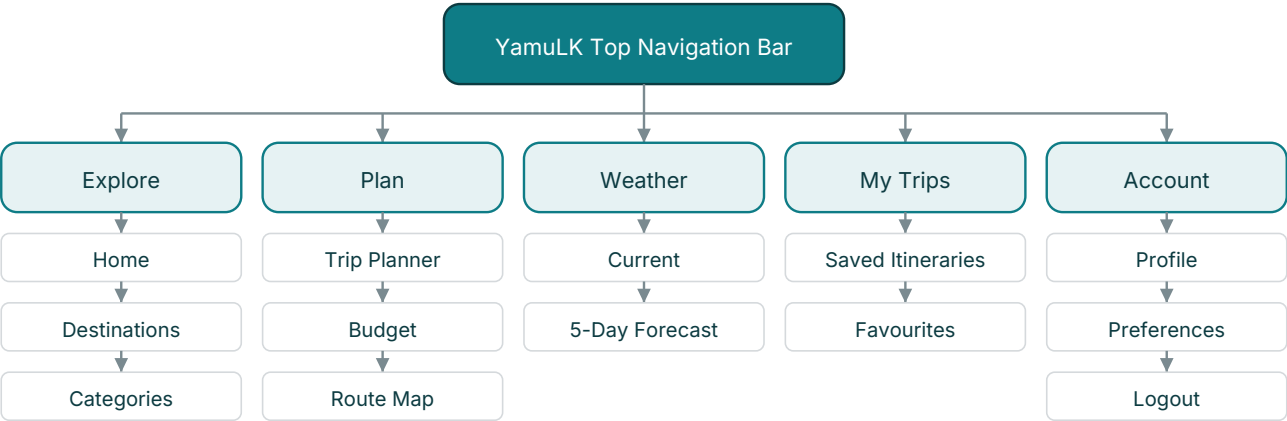


Figure 1 — Two-level menu hierarchy of the top navigation bar. Each group expands into its child items on hover or click.

2.2 Menu Inventory

Menu	Menu items	Visible to	Action triggered
Top Navigation Bar	Home, Destinations, Trip Planner, Budget, Route, Weather, Saved Trips	Guest and registered user (Saved Trips requires login)	Client-side route change through React Router; no full page reload.
Account Dropdown	My Profile, Preferences, Saved Trips, Help, Logout	Registered user only	Opens the profile page, settings panel, or clears the JWT and returns to Login.
Authentication Menu	Login, Register, Continue as Guest	Unauthenticated visitor	Submits credentials, opens the registration form, or starts a temporary guest session.
Category Filter Menu	Beaches, Waterfalls, Mountains, Historical Places, All	Everyone	Filters the destination grid by category using a query parameter.
Search and Sort Menu	Search box, Sort by rating / cost, Province filter, Budget range	Everyone	Sends query parameters to the search endpoint and re-renders the grid.
Destination Action Menu	Plan Trip, Save to Favourites, Share, Write Review	Plan Trip and Save require login	Opens the trip planner, writes a saved item, copies the share link, or opens the review form.
Saved Trips Tab Menu	Saved Itineraries, Favourite Destinations	Registered user only	Switches the data set shown in the card grid without leaving the page.
Trip Context Menu	Edit Trip, Duplicate, Export as PDF, Delete	Trip owner only	Performs the corresponding CRUD operation on the selected itinerary.
Footer Menu	About, Contact, Privacy Policy, Terms, Social links	Everyone	Opens static informational pages.

3. GUI Elements

The inventory below maps every required control type to the interfaces where it appears and to the React component that implements it. Grouping controls into reusable components keeps the interface consistent and reduces duplicated code across the ten pages.

3.1 Element Inventory by Interface

GUI element	Where it appears	Purpose	React component
Text fields	Login, Register, Search bar, Trip Planner, Budget Calculator, Profile	Free text and numeric entry for credentials, destination search, budget and expense values.	<InputField />
Password fields	Login, Register, Change Password	Masked credential entry with a show/hide toggle.	<PasswordField />
Buttons	All ten pages	Primary actions such as Login, Generate Plan, Calculate, Start Navigation, Save and Share.	<Button variant />
Labels	All forms and info panels	Identify each field and display read-only values such as distance and estimated time.	<Label />
Tables	Budget Calculator breakdown, Saved Trips list view, Admin destination list	Display itemised rows of expenses and trips with sortable headers.	<DataTable />
Checkboxes	Remember Me, Preferences, Include transport / meals in budget, Filter options	Multi-select boolean options.	<Checkbox />
Radio buttons	Transport method (Bus, Train, Car, Bike), Trip type	Mutually exclusive single choice.	<RadioGroup />
Dropdowns	Province, Sort by, Number of people, Currency	Select one value from a known list.	<Select />
Date pickers	Trip Planner travel date and return date	Calendar-based date selection with validation.	<DatePicker />
Cards	Dashboard recommendations, Saved Trips, Favourites	Grouped preview of a destination or itinerary with image, rating and cost.	<DestinationCard />
Tabs	Saved Trips, Destination Details information panel	Switch between related data sets in place.	<Tabs />
Image gallery	Destination Details	Hero image with selectable thumbnails.	<Gallery />
Rating widget	Destination Details, Review form	Show average stars and capture a new rating.	<StarRating />
Map widget	Route Planner, Destination Details mini-map	Embed the Google Map with start, end and route polyline.	<MapView />
Status badges	Budget Calculator, Trip cards	Colour-coded state such as Within Budget or Over Budget.	<Badge />
Progress and loaders	Splash screen, all API calls	Communicate loading state during data fetches.	<Spinner /> , <Skeleton />

GUI element	Where it appears	Purpose	React component
Modals and toasts	Confirm delete, Share link, Save success	Confirmation dialogs and transient feedback messages.	<Modal /> , <Toast />
Navigation bar	Every authenticated page	Persistent top-level menu and account dropdown.	<Navbar />

3.2 Component and Interaction Standards

- Reusable components. Every element above is written once as a functional React component with props, then reused across pages, so a change to a button style applies everywhere.
- Controlled inputs. All form fields are controlled components whose value lives in React state, which allows live validation, budget recalculation as the user types and instant enable or disable of the submit button.
- Validation feedback. Invalid fields show a red border, an inline message below the field and an icon, so the error is never signalled by colour alone.
- Loading and empty states. Every data-driven area has three visual states — loading skeleton, loaded content and an empty-state message with a call to action.
- Accessibility. All controls carry labels and ARIA attributes, are reachable by keyboard tab order, and text meets a 4.5:1 contrast ratio.
- Responsive behaviour. Grids collapse from four columns to two and then one at 1024 px and 768 px breakpoints; the top navigation collapses into a compact menu button on narrow screens.

4. Navigation Design

Navigation follows a hub-and-spoke model. The Home Dashboard is the hub; every functional area is reachable from it in one click, and the persistent top navigation bar means the user is never more than one click away from any main section. Routing is handled client-side by React Router, so transitions are instant and the browser back button, bookmarking and deep links all work as users expect on the web.

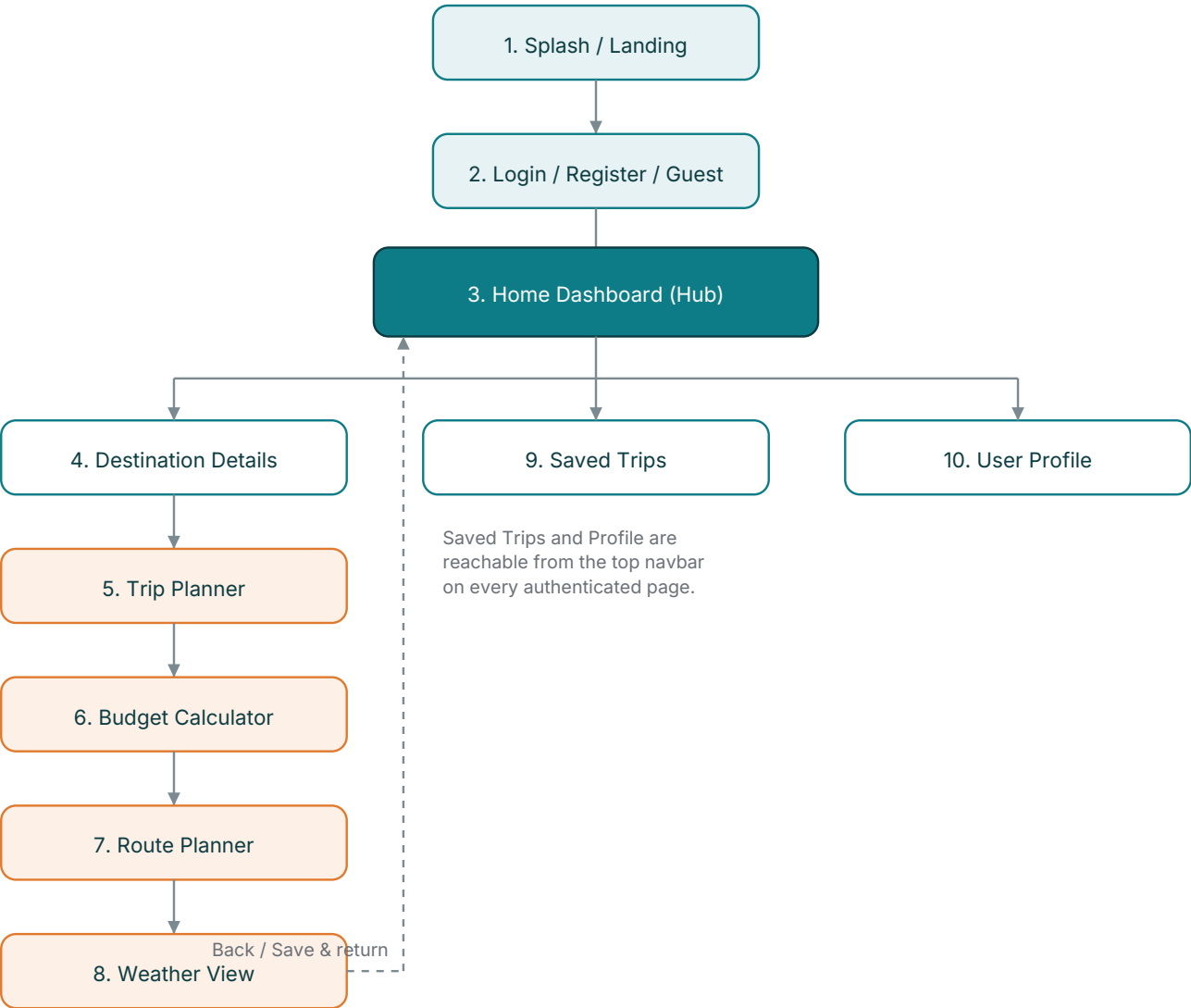


Figure 2 — Screen-to-screen navigation flow. Solid arrows are forward navigation; the dashed path is the return route to the hub.

4.1 Route Map

URL route	Screen	Access	Reached from
/	Splash / Landing	Public	Application entry point
/login	Login, Register, Guest	Public	Splash, Logout, any protected route
/dashboard	Home Dashboard (hub)	Guest or user	Login, Guest entry, navbar logo
/destination/:id	Destination Details	Guest or user	Recommendation card, search result
/plan/:id	Trip Planner	User	Plan Trip button, navbar
/plan/:id/budget	Budget Calculator	User	Generate Plan result

URL route	Screen	Access	Reached from
/plan/:id/route	Route Planner	Guest or user	Budget page, navbar
/weather/:id	Weather View	Guest or user	Route page, destination panel
/saved	Saved Trips and Favourites	User	Navbar, account dropdown
/profile	User Profile and Preferences	User	Account dropdown

4.2 Navigation Rules

- Protected routes. /plan, /saved and /profile are wrapped in a route guard that checks the JWT; an unauthenticated visitor is redirected to /login and returned to the intended page after a successful login.
- Guest limits. A guest can browse, search, view details, check routes and weather, but attempting to save or plan a trip opens a login prompt modal.
- Browser history. Every navigation pushes a history entry, so the back and forward buttons move between screens correctly and every screen has a shareable URL.
- Breadcrumbs. Deep pages show a breadcrumb trail such as Home / Destinations / Ella / Trip Planner for orientation.
- Persistent shell. The navigation bar and footer are rendered once in the root layout component; only the page area re-renders on route change.

5. Back-End Plan

The back end is an Express.js REST API running on Node.js, backed by MongoDB through Mongoose. It is stateless: each request carries a JSON Web Token, so the API can be scaled horizontally without sticky sessions.

5.1 Required Data and Collections

Collection	Key fields	Notes
users	_id, name, email (unique), passwordHash, preferences {categories, currency, homeProvince}, savedTrips [ObjectId], createdAt	Passwords hashed with bcrypt; email carries a unique index.
destinations	_id, name, category, province, description, images [url], location {lat, lng}, averageRating, estimatedCost, popularity	Text index on name and description for search; 2dsphere index on location.
trips	_id, userId, destinationId, travelDate, people, transportMode, itinerary [{day, activities}], budgetId, status	One trip document per planned journey; references the user and the destination.
budgets	_id, tripId, transport, accommodation, food, other, totalCost, userBudget, status	Status is a derived value: Within Budget, Near Limit or Over Budget.
saveditems	_id, userId, itemType (destination or trip), itemId, savedAt	Compound unique index on userId and itemId prevents duplicates.
reviews	_id, userId, destinationId, rating, comment, createdAt	Aggregation pipeline recalculates the destination average rating.
weathercache	_id, destinationId, fetchedAt, current, forecast [5 days]	TTL index expires documents after 30 minutes to limit external API calls.

5.2 Major Back-End Functions and API Endpoints

Service module	Endpoint	Function	Description
Auth	POST /api/auth/register	registerUser()	Validates input, hashes the password and creates the user document.
Auth	POST /api/auth/login	loginUser()	Verifies credentials and returns a signed JWT.
Auth	GET /api/auth/me	getCurrentUser()	Returns the profile of the token holder.
Destination	GET /api/destinations	listDestinations()	Supports search, category, province and sort query parameters with pagination.
Destination	GET /api/destinations/:id	getDestinationById()	Returns full detail plus nearby hotels, restaurants and attractions.
Trip	POST /api/trips	generateItinerary()	Builds a day-by-day plan from date, party size, budget and transport mode.
Trip	GET /api/trips	getUserTrips()	Lists all itineraries belonging to the logged-in user.
Trip	PUT /api/trips/:id	updateTrip()	Edits an existing itinerary.
Trip	DELETE /api/trips/:id	deleteTrip()	Removes an itinerary and its budget document.

Service module	Endpoint	Function	Description
Budget	POST /api/budget/calculate	calculateBudget()	Sums the expense categories and returns the total.
Budget	GET /api/budget/:tripId	checkBudgetStatus()	Compares total cost against the user budget and returns the status label.
Route	GET /api/route	getRoute()	Calls the Google Maps Directions API and returns distance, duration and polyline.
Weather	GET /api/weather/:destId	getWeather()	Returns cached or freshly fetched current conditions and a five-day forecast.
Saved	POST /api/saved	saveItem()	Adds a destination or trip to the user's saved list.
Saved	DELETE /api/saved/:id	removeSavedItem()	Removes an entry from the saved list.
Profile	PUT /api/users/profile	updateProfile()	Updates name, preferences and password.
Review	POST /api/reviews	addReview()	Stores a review and recalculates the destination average rating.

5.3 Database Requirements

- Database type. MongoDB, a document database, is used because destination and itinerary records are nested and vary in shape, which maps directly onto JSON handled by React and Express.
- Schema enforcement. Mongoose schemas apply types, required fields and enumerations at the application layer, giving structure without losing document flexibility.
- Relationships. References with ObjectId link trips to users and destinations; population is used on read. Small, frequently read values such as destination name and thumbnail are embedded in trip documents to avoid extra queries.
- Indexing. Unique index on user email, text index on destination name and description, 2dsphere index on coordinates for nearby searches, and a TTL index on the weather cache.
- Guest sessions. Guest activity is held in browser local storage only and is merged into the account if the visitor later registers.
- Security. Passwords stored as bcrypt hashes, JWT expiry of 24 hours, server-side validation on every endpoint, CORS restricted to the client origin, and API keys kept in server environment variables.
- Backup and scale. MongoDB Atlas provides automated daily snapshots and replica sets; the stateless API allows horizontal scaling behind a load balancer.

5.4 Example Mongoose Schema

```
const tripSchema = new mongoose.Schema({
  userId: { type: ObjectId, ref: 'User', required: true },
  destinationId: { type: ObjectId, ref: 'Destination', required: true },
  travelDate: { type: Date, required: true },
  people: { type: Number, min: 1, default: 1 },
  transportMode: { type: String, enum: ['bus', 'train', 'car', 'bike'] },
  itinerary: [{ day: Number, activities: [String] }],
  budget: { transport: Number, accommodation: Number, food: Number, other: Number },
}, { timestamps: true });
```

6. Overall System Model

YamuLK follows a three-tier client-server architecture with an additional external services tier. The React client never talks to the database directly: every data operation passes through the Express API, which owns validation, authentication and business rules. This separation keeps API keys off the browser and allows the client and server to be deployed and scaled independently.

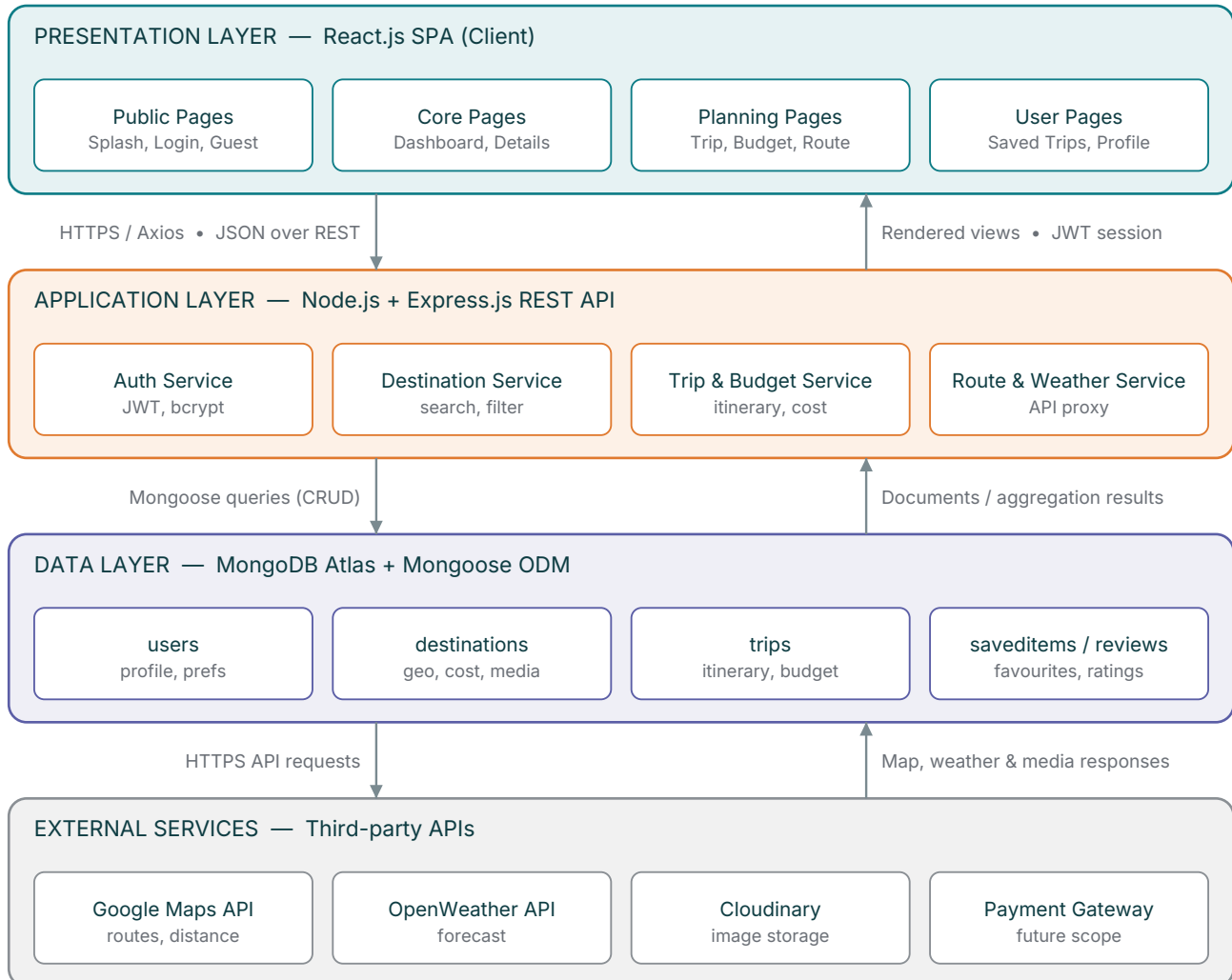


Figure 3 — Overall MERN system model showing the four tiers and the data exchanged between them.

6.1 Layer Responsibilities

Tier	Components	Responsibility
Presentation (Client)	React.js SPA, React Router, Axios, Context or Redux state	Renders the ten interfaces, validates input on the client side, stores the JWT and re-renders on state change.
Application (Server)	Node.js runtime, Express.js routes, controllers, middleware, service modules	Authenticates requests, validates payloads, executes itinerary and budget logic, and proxies external APIs so keys stay on the server.
Data	MongoDB Atlas cluster, Mongoose models, indexes	Persists users, destinations, trips, budgets, saved items and reviews, and serves aggregation queries.
External services	Google Maps, OpenWeather, Cloudinary	Provide routing, distance, forecast and image hosting that the application does not generate itself.

6.2 Request Lifecycle Example — Generating a Trip Plan

Step	Action
1	The user fills the Trip Planner form and clicks Generate Plan in the React client.
2	Axios sends POST /api/trips with the form payload and the JWT in the Authorization header.
3	Express middleware verifies the token and validates the request body.
4	The trip service reads the destination document from MongoDB and calls the Maps and Weather services for distance and forecast.
5	generateltinerary() builds the day plan and calculateBudget() produces the cost breakdown.
6	The trip and budget documents are written to MongoDB and the combined result is returned as JSON.
7	React stores the response in state and renders the Budget Calculator screen without a page reload.

7. Conclusion

The menu structure, GUI inventory, navigation model, back-end plan and system model described here form a complete and internally consistent design for YamuLK as a browser-based web application on the MERN stack. Every screen defined in the interface design maps to a React route, every route maps to one or more Express endpoints, and every endpoint maps to a MongoDB collection, so implementation can begin directly from this document.